

SQL応用講座4

SQL講座(Transaction)

トランザクション・・・複数の処理をまとめたもので、コミットかロールバックをするまで処理を完了せず、削除、UPDATEなどをするするとテーブルをロックして、他のプロセスからテーブルを削除・更新できないようにする

BEGIN TRANSACTION

トランザクションを開始する

COMMIT

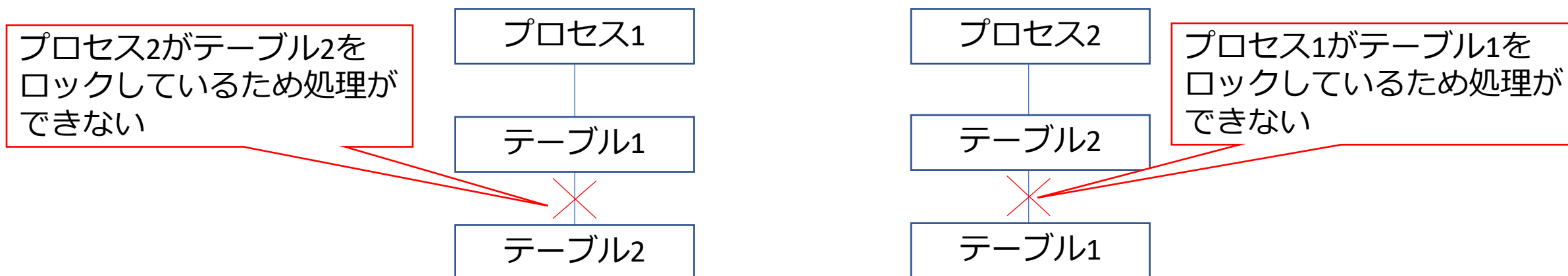
トランザクション中に実行した処理を反映する

ROLLBACK

トランザクション中の実行結果を破棄してトランザクション前に戻す

DEAD LOCK

複数のプロセスが互いのテーブルをロックしてその次の処理にすすめなくなること



SQL講座(View)

Viewとは

SQLを格納して、Viewからデータを取り出す際にSQLを実行して取り出すことができる

以下の場面で利用する

- ・ 抽象的なテーブルを用意して、外部からアクセスしやすい形にする
- ・ 複雑なSQLをViewの中に入れて、データへのアクセスをカプセル化して簡単に実行できるようにする

Viewの作成

```
CREATE VIEW [IF NOT EXISTS]
view_name[(column-name-list)]
AS
    select-statement;
```

Viewの結果の取得

```
SELECT
    *
FROM
    view_name
```

Viewの削除

```
DROP VIEW view_name
```

SELECTの結果をv_tracksというViewに入れる

```
CREATE VIEW v_tracks
AS
SELECT
    trackid,
    tracks.name,
    albums.Title AS album,
    media_types.Name AS media,
    genres.Name AS genres
FROM
    tracks
    INNER JOIN albums ON Albums.AlbumId = tracks.AlbumId
    INNER JOIN media_types ON media_types.MediaTypeId =
        tracks.MediaTypeId
    INNER JOIN genres ON genres.GenreId = tracks.GenreId;
```

SQL講座(SQLチューニング(実行計画の見方))

SQLiteの実行計画の見方について説明します

```
sqlite> EXPLAIN QUERY PLAN SELECT a, b FROM t1 WHERE a=1;  
QUERY PLAN  
`--SCAN TABLE t1
```

→ **SCAN TABLE t1**だけの場合、FULL SCANのことでテーブルの全行の中身を確認して必要なものだけに絞り込む

```
sqlite> CREATE INDEX i1 ON t1(a);  
sqlite> EXPLAIN QUERY PLAN SELECT a, b FROM t1 WHERE a=1;  
QUERY PLAN  
`--SEARCH TABLE t1 USING INDEX i1 (a=?)
```

→ **USING INDEX**とは、INDEX SCANのことでINDEXを利用して必要なデータを取得している

```
sqlite> CREATE INDEX i2 ON t1(a, b);  
sqlite> EXPLAIN QUERY PLAN SELECT a, b FROM t1 WHERE a=1;  
QUERY PLAN  
`--SEARCH TABLE t1 USING COVERING INDEX i2 (a=?)
```

→ 検索対象がインデックスだけで完結する場合（例では、a, b）、表からデータを読み取ることなく、検索はインデックスだけで完結できる。そのため、より高速に検索できる

SQL講座(SQLチューニング(実行計画の見方))

ORDER BYの対象カラムにインデックスを追加して、処理を高速化する

```
sqlite> EXPLAIN QUERY PLAN SELECT c, d FROM t2 ORDER BY c;  
QUERY PLAN  
|--SCAN TABLE t2  
`--USE TEMP B-TREE FOR ORDER BY
```

→t2からデータを取得して、一時的にB-TREEを用いて並び替える

```
sqlite> CREATE INDEX i4 ON t2(c);  
sqlite> EXPLAIN QUERY PLAN SELECT c, d FROM t2 ORDER BY c;  
QUERY PLAN  
`--SCAN TABLE t2 USING INDEX i4
```

→INDEXを用いて、テーブルを並べ替えるため、高速になる

SQL講座(SQLチューニング(基本))

1. *ですべてのカラムからデータを取得しない。必要なカラムだけ取ってくる

×悪い例

```
SELECT * FROM User
```

○良い例(必要なカラムだけ指定して取ってくる)

```
SELECT id, name, age FROM User
```

2. どのテーブルから取得したカラムなのかわかるようにテーブルに別名を付ける
(別名がない場合、どちらのテーブルからなのかDBが解析するため、時間がかかる)

×悪い例

```
SELECT user_id, user_name, age, school_name  
FROM Users, Schools WHERE school_id = school_id
```

○良い例

```
SELECT u.user_id, u.user_name, u.age, s.school_name  
FROM Users AS u, Schools AS s WHERE u.school_id =  
s.school_id
```

SQL講座(SQLチューニング(処理の無駄をなくす))

1. HAVINGでなく、WHEREを付けて絞り込む

(HAVINGだと、その前段のGROUP BYですべてのデータを集計する必要があり、時間がかかる。WHEREだと絞り込んだ後にGROUP BYするので、集計する量が少なく時間が短い(ただし、sqliteの場合自動的に絞り込まれる))

×悪い例

```
SELECT id, product_name, SUM(amounts)
FROM SALES_HISTORY AS sh
GROUP BY sh.id, sh.product_name
HAVING sh.id=10
```

○良い例

```
SELECT id, product_name, SUM(amounts)
FROM SALES_HISTORY AS sh
WHERE sh.id=10
GROUP BY sh.id, sh.product_name
```

2. 副問い合わせでのSELECTの実行回数はなるべく少なくする

×悪い例

```
SELECT id, product_name
FROM SALES_HISTORY AS sh
WHERE sh.id
IN(SELECT id FROM Product WHERE category='Mobile')
AND
sh.id
IN(SELECT id FROM Product WHERE color='BLACK')
```

○良い例

```
SELECT id, product_name
FROM SALES_HISTORY AS sh
WHERE sh.id
IN(SELECT id FROM Product WHERE category='Mobile'
AND color='BLACK')
```

SQL講座(SQLチューニング(INDEXを利用する))

1. INDEXを作成する

→検索対象、GROUP BY, ORDER BYの対象カラムにインデックスを作成して高速化する

2. 関数INDEX

→関数の実行結果に対して、検索をかけている場合は、関数とセットでインデックスを作成する
CREATE INDEX 関数()

3. INDEXが利用できない場合

3-1. 後方一致、中間一致

後方一致と中間一致はインデックスが利用されない(SQLiteは前方一致もインデックスが利用されない)
前方一致: name LIKE 'A%', 後方一致: '%a', 中間一致: '%a%'

3-2. 型が異なる場合

文字列型のカラムnameを数値と比較するなど
×悪い例: code = 10, ○良い例: code = '10'

3-3. 計算式を利用している場合

×悪い例

列に計算式を追加するとインデックスを取得して絞り込みができない

SELECT * FROM Users WHERE age + 4 > 30

○良い例

SELECT * FROM Users WHERE age > 26

SQL講座(SQLチューニング(IN, EXISTS))

1. 外側のテーブルのデータ量が多く、内側(結合先)のテーブルのデータ量が少ない時はINを利用する
2. 外側のテーブルのデータ量が少なく、内側(結合先)のテーブルのデータ量が多い時はEXISTSを利用する

***) 但し、インデックスを活用できていることが前提**

INを利用した場合

```
SELECT
*
FROM students
WHERE school_no IN (
  SELECT school_no FROM schools);
```

外側のテーブル: students
内側のテーブル: schools
実行の際には、schoolsからschool_noを取得して、studentsテーブルのschool_noに存在するものだけに絞り込む。
school(内側のテーブル)の方がデータ量が少ないほうが、絞り込む回数が少なく速い

EXISTSを利用した場合

```
SELECT
*
FROM students
WHERE EXISTS (
  SELECT 1 FROM schools where
  students.school_no = schools.school_no);
```

外側のテーブル: students
内側のテーブル: schools
実行の際には、studentsからデータを取得して、school_no同士で紐づけて絞り込む。
student(外側のテーブル)の方がデータ量が少ないほうが、絞り込む回数が少なく速い

NOT INはインデックスを利用できないため、遅いので注意